

第 32 讲

Dataset、DataLoader 与 Mini-Batch

数据到底怎样一批一批进入 Training Loop

《面向 LLM / Agent Systems 科研的深度学习零基础课》

【本讲定位】第 31 讲已经把一个完整 PyTorch Training Step 拆成 `zero_grad -> forward -> loss -> backward -> step`。但当时我们把整个 Tensor `X,Y` 一次性交给模型，真实科研训练通常不是这样。第 32 讲解决的是：数据如何被组织成 Sample, Dataset 如何定义“有哪些样本”，DataLoader 如何按规则取样、打乱并组成 Mini-Batch，以及 Epoch / Batch / Step 到底怎样精确对应。

学习目标

学完本讲，你应该能够：

1. 区分 Sample、Feature、Label、Dataset、Batch、Mini-Batch；
2. 理解 Dataset 为什么只负责“第几个样本是什么”，而不负责训练；
3. 理解 DataLoader 为什么是“取样规则 + 组 Batch + 迭代接口”；
4. 使用 TensorDataset 和自定义 Dataset；
5. 解释 `batch_size`、`shuffle`、`drop_last`；
6. 准确计算一个 Epoch 有多少 Batch、多少 Optimizer Step；
7. 写出 `for epoch ... for xb,yb in loader ...` 的标准双层训练循环；
8. 根据任务预测 Batch Tensor 的 Shape；
9. 理解 LLM 中 `input_ids:(B,T)`、`labels:(B,T)` 的数据来源；
10. 识别科研代码中常见的数据管线错误：Data Leakage、错误 shuffle、错误 loss 平均、最后一个 Batch Shape 假设等。

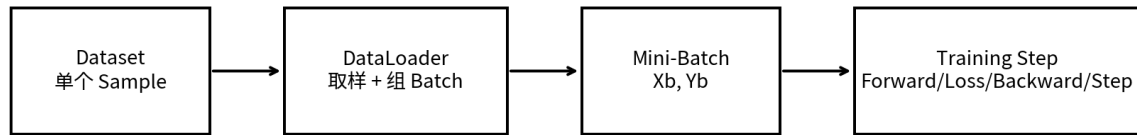
1 先把整个数据流放在脑中：DataLoader 在训练闭环的哪里？

第 31 讲的训练循环写成：

$X,Y \rightarrow \text{Forward} \rightarrow \text{Loss} \rightarrow \text{Backward} \rightarrow \text{Step}$

但真实数据集可能有 10 万、100 万甚至更多样本，不可能总把全部样本一次送入模型。因此更现实的结构是：

Dataset $\rightarrow$ DataLoader $\rightarrow$ Mini-Batch $\rightarrow$ Training Step



一个 Epoch = DataLoader 被完整遍历一遍

图 1: Dataset $\rightarrow$ DataLoader $\rightarrow$ Mini-Batch $\rightarrow$ Training Step

DataLoader 不是模型，也不是 Optimizer。它属于“数据输入管线”。它的工作是：从 Dataset 取出若干 Sample，按规则组成一个 Batch，然后把这个 Batch 交给训练代码。

2 【原理】什么叫一个 Sample?

假设做最简单的回归：根据一个输入数值预测输出：

$$y = 3x - 2$$

我们准备 10 个样本：

$$(x_0, y_0), (x_1, y_1), \dots, (x_9, y_9)$$

其中一个 Sample 可以写成：

$$(x, y) = ([5], [13])$$

这里：

- x ：输入 Feature（特征）；
- y ：监督学习里的 Target / Label（目标）；
- (x, y) ：一个 Sample（样本）。

若整个数据：

$$X.shape = (10, 1), \quad Y.shape = (10, 1)$$

这里第一个维度 10 表示有 10 个 Sample。

Sample 不是一行数字

一个 Sample 具体是什么，取决于任务：

任务	一个 Sample 的输入	一个 Sample 的目标
表格回归	一行数值特征	一个连续值
图像分类	一张图像 (C, H, W)	一个类别 ID
文本分类	一段 token IDs	一个类别 ID
LLM 预训练	一段 token 序列	右移后的 next-token labels
Agent 离线轨迹	一段状态/动作/工具 Trace	任务标签、回报或下一步目标

科研里看到 “dataset size = 50k” 时，先问清楚：这里的 50k 到底是 50k 个图像、50k 条文本、50k 个 token、50k 个 trajectory，还是 50k 个 pair？“Sample 的定义” 会直接影响统计单位和实验解释。

3 【通俗解释】为什么需要 Dataset 抽象？

最简单时，我们当然可以直接写：

```
X = torch.tensor(...)
Y = torch.tensor(...)
```

然后用 $X[i], Y[i]$ 取样本。

但真实科研数据常常来自：

- 图片文件；
- JSON / JSONL；
- CSV / Parquet；
- 文本文件；
- tokenized cache；
- Agent trajectory / trace；
- 多个字段组合的数据结构。

因此 PyTorch 希望我们统一回答两个问题：

1. 这个 Dataset 一共有多少个 Sample？
2. 给我索引 idx，第 idx 个 Sample 到底是什么？

这就是：

```
def __len__(self):
    ...

def __getitem__(self, idx):
    ...
```

Dataset 的核心不是“把所有数据都塞进一个 Tensor”。它更像一份“样本访问协议”：只要能够回答 `len(dataset)` 和 `dataset[idx]`，DataLoader 就知道怎样从中取样。

4 TensorDataset：已经有 Tensor 时的最小 Dataset

如果输入和标签已经是 Tensor：

```
X = torch.arange(10, dtype=torch.float32).reshape(-1, 1)
Y = 3 * X - 2

dataset = TensorDataset(X, Y)
```

那么：

```
len(dataset)
```

返回 10，而：

```
x, y = dataset[3]
```

返回第 3 个样本。

本讲实际输出：

```
len(TensorDataset) = 10
basic_dataset[3] -> x = [3.0] y = [7.0]
```

TensorDataset(X,Y) 默认认为这些 Tensor 的第 0 维是 Sample 维。因此 X 和 Y 的第 0 维长度必须一致。

5 自定义 Dataset：__len__ 与 __getitem__ 真正干什么？

最小自定义版本：

```
class ToyRegressionDataset(Dataset):
    def __init__(self, n=12):
        self.x = torch.linspace(-1, 1, n).reshape(-1, 1)
        self.y = 3 * self.x - 2

    def __len__(self):
        return len(self.x)

    def __getitem__(self, idx):
        return self.x[idx], self.y[idx]
```

逐项理解：

- `__init__`: 准备路径、索引、Tensor、Tokenizer 等资源;
- `__len__`: 告诉外界 Dataset 长度;
- `__getitem__`: 定义 “一个 Sample 长什么样”。

例如图像任务的 `__getitem__` 可能做:

$\text{idx} \rightarrow \text{读取图像} \rightarrow \text{resize/normalize} \rightarrow (\text{image}, \text{label})$

文本任务则可能做:

$\text{idx} \rightarrow \text{读取文本} \rightarrow \text{Tokenizer} \rightarrow (\text{input_ids}, \text{label})$

Dataset 是科研可复现性的一个关键位置。如果训练集和测试集的数据预处理、过滤规则、tokenization、截断方式不一致，结果可能改变。论文的 “Data preprocessing” 不是工程细节，而是实验定义的一部分。

6 【原理】什么是 Batch / Mini-Batch?

如果 Dataset 有:

$$N = 10$$

个 Sample, 设:

$$\text{batch_size} = 4$$

那么可以分成:

$$[0, 1, 2, 3]$$

$$[4, 5, 6, 7]$$

$$[8, 9]$$

因此一共有:

$$\left\lceil \frac{10}{4} \right\rceil = 3$$

个 Batch。

$N=10, \text{batch_size}=4, \text{drop_last}=False$

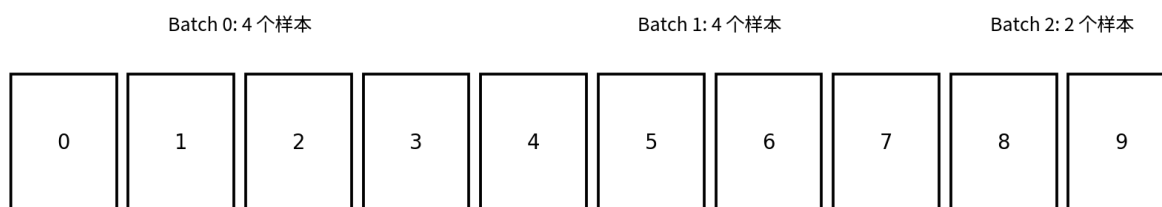


图 2: $N=10$ 、 $\text{batch_size}=4$ 时的 Batch 划分

为什么叫 Mini-Batch?

三个概念要区分：

- Full Batch：一次使用全部 N 个样本；
- Single-sample / 经典 SGD：一次一个样本；
- Mini-Batch：一次用一小批样本，例如 32、64、128。

现代深度学习通常说“SGD 训练”，工程上常常实际使用 Mini-Batch SGD。

为什么现实中普遍使用 Mini-Batch?

最低层有三个理由：

1. **内存限制**：整个数据集可能根本放不进 GPU；
2. **矩阵并行效率**：一次计算一批样本比一个样本一个样本更适合 GPU；
3. **更新频率**：不必等整个数据集算完才更新一次参数。

Batch Size 不是“一个无关紧要的加载参数”。它会影响显存、每个 Epoch 的 Step 数、Gradient 的统计性质和训练动力学。后面第 34–35 讲还会继续深入。

7 【具体数字例子】手工切 Batch 和 DataLoader 有什么区别?

不用 DataLoader，我们可以自己写：

```
for start in range(0, len(X), 4):
    xb = X[start:start+4]
    yb = Y[start:start+4]
```

实验得到：

```
start=0 -> [0,1,2,3]
start=4 -> [4,5,6,7]
start=8 -> [8,9]
```

DataLoader：

```
loader = DataLoader(
    dataset,
    batch_size=4,
    shuffle=False,
    drop_last=False,
)
```

输出完全一致。

因此 DataLoader 并没有发明新的数学。它把“索引哪些样本、怎样分批、是否打乱、怎样并行读取、怎样组装 Batch”统一封装起来。

8 DataLoader 到底做了什么？

一个简化心智模型：

选择一组 indices $\rightarrow dataset[idx] \rightarrow$ 收集 Sample $\rightarrow$ collate 成 Batch

例如索引：

$[4, 5, 7, 2]$

DataLoader 会近似地执行：

$dataset[4], dataset[5], dataset[7], dataset[2]$

然后把每个 Sample 中同类字段堆叠起来：

$(x_4, x_5, x_7, x_2) \rightarrow X_b$

$(y_4, y_5, y_7, y_2) \rightarrow Y_b$

因此循环：

```
for xb, yb in loader:
    ...
```

每次拿到的 xb, yb 已经是一个 Batch。

9 【Shape】Batch 维是怎样出现的？

假设单个回归样本：

$x.shape = (1,)$

如果 Batch Size=8：

$X_b.shape = (8, 1)$

图像：单张图：

(C, H, W)

Batch 后：

(B, C, H, W)

LLM token IDs：单条序列：

(T)

Batch 后：

(B, T)

Embedding 后：

(B, T, D)

第 0 维常常就是 Batch 维。Dataset 的 `getitem` 通常返回“单个样本 Shape”，DataLoader collate 后才自动在前面堆出 Batch 维。

10 shuffle=True: 到底打乱了什么?

本讲实验对 0-9 十个样本进行 shuffle:

```
shuffle order #1 = [4,5,7,2,8,3,1,0,9,6]
```

排序以后仍然是:

$0, 1, 2, \dots, 9$

所以 shuffle 改变的是:

样本访问顺序

不是样本内容, 也不是增加/删除样本。

Shuffle 改变样本访问顺序, 但样本集合没有改变

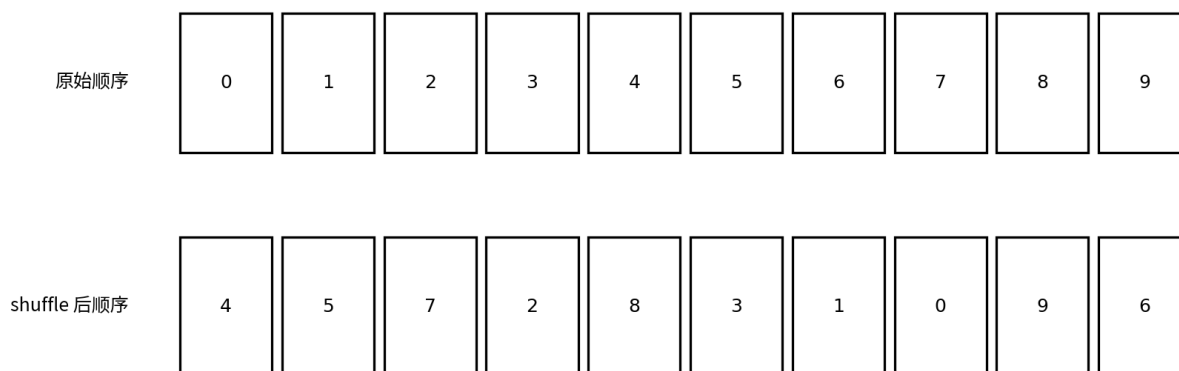


图 3: Shuffle 改变顺序, 但没有改变样本集合

为什么训练常 shuffle?

如果数据原本按类别、时间、来源排好, 连续 Mini-Batch 可能非常偏。例如前 1000 条全是类别 A, 后 1000 条全是类别 B。训练时打乱可以让每个 Batch 更接近总体混合分布。

Validation / Test 要不要 shuffle?

通常:

- Training: 常用 shuffle=True;
- Validation/Test: 通常 shuffle=False, 方便可复现与逐样本追踪。

数学上, 如果只计算顺序无关的整体平均指标, 测试集 shuffle 通常不会改变最终指标; 但它没有太大必要, 还会降低调试便利性。

时间序列、因果序列或某些 episode 数据不能机械 shuffle。是否允许打乱, 取决于“样本是否独立”和任务定义。不要把图像分类里的默认习惯机械搬到所有任务。

11 随机性与可复现：同样是 shuffle，为什么每次顺序不同？

Shuffle 本身要使用随机数。为了复现实验，可以：

```
g = torch.Generator().manual_seed(3201)
loader = DataLoader(
    dataset,
    batch_size=4,
    shuffle=True,
    generator=g,
)
```

这里 Seed 不是“让模型更准确”，而是让随机过程更容易复现。

论文比较两个方法时，至少应注意数据划分、shuffle seed、模型初始化 seed 是否形成公平对照。否则“方法差异”可能混进随机初始化和样本顺序差异。后面实验可复现性课程会继续系统处理。

12 drop_last：最后不足一个 Batch 的样本怎么办？

还是：

$$N = 10, \quad B = 4$$

drop_last=False:

[4, 4, 2]

三个 Batch，最后两个样本仍然参与。

drop_last=True:

[4, 4]

最后两个样本被丢弃。

实验实际输出：

```
drop_last=False batch sizes = [4, 4, 2]
drop_last=True  batch sizes = [4, 4]
```

什么时候可能用 drop_last=True?

某些训练过程希望所有 Batch Shape 完全一致，或者某些 Batch-dependent 操作对极小最后 Batch 不稳定时，可能选择丢弃。

什么时候不应该随便丢？

Validation/Test 通常更希望评估所有样本。因此常用：

```
drop_last=False
```

如果数据集很小，随便 `drop_last=True` 可能每个 Epoch 都永久丢掉一部分样本。训练时是否可接受，要看 sampler/shuffle 机制和研究目的；评估阶段尤其不要无意识丢样本。

13 Epoch、Batch、Step：最容易混淆的三个词

这是本讲必须真正掌握的部分。

Epoch

1 Epoch = Dataset 被完整遍历一遍

注意：“完整遍历”是概念定义；如果主动 `drop_last=True`，代码实际可能不使用最后不足 Batch 的样本。

Batch

一次从 DataLoader 拿到的一组 Sample。

Step

在普通训练代码里，通常指一次 Optimizer Update：

$1 \text{ Step} \approx 1 \text{ Mini-Batch Forward} + \text{Backward} + \text{Optimizer Update}$

如果：

$$N = 23, \quad B = 6$$

且 `drop_last=False`：

$$\text{batches per epoch} = \left\lceil \frac{23}{6} \right\rceil = 4$$

Batch Size 实际为：

$$[6, 6, 6, 5]$$

训练 3 个 Epoch：

$$3 \times 4 = 12 \text{ steps}$$

本讲实验实际得到：

```
epoch=0 ... cumulative_steps=4
epoch=1 ... cumulative_steps=8
epoch=2 ... cumulative_steps=12
```

14 把 DataLoader 接回第 31 讲 Training Loop

真实代码现在变成双层循环：

```
for epoch in range(num_epochs):  
    for xb, yb in train_loader:  
        optimizer.zero_grad()  
        pred = model(xb)  
        loss = loss_fn(pred, yb)  
        loss.backward()  
        optimizer.step()
```

外层:

Epoch

内层每一次:

Mini-Batch Step

本讲训练:

$N = 80, \quad B = 8$

所以:

$len(loader) = 10$

每 Epoch 有 10 个 Step; 20 Epoch:

$20 \times 10 = 200 \text{ Steps}$

实验实际:

```
总 global_step = 200  
理论总 step = 200
```

最终 Linear 模型学到:

$weight = 3.0, \quad bias \approx -2.0$

对应数据真实规律:

$y = 3x - 2$

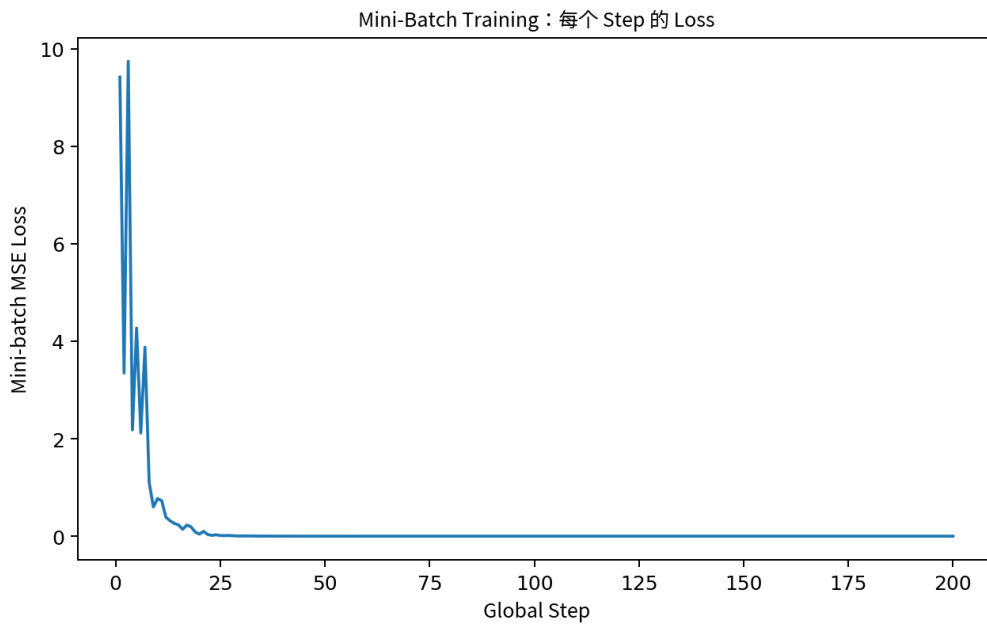


图 4: 每一个 Mini-Batch Step 的 Loss: 会有波动, 但总体下降

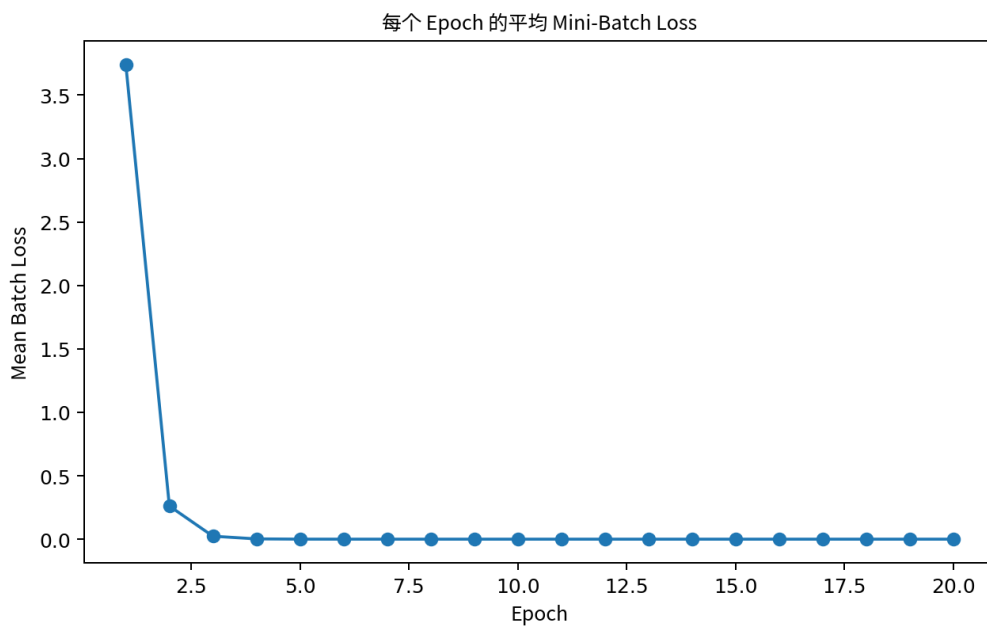


图 5: 按 Epoch 汇总后的平均 Loss 更平滑

15 为什么 Mini-Batch Loss 会抖动?

Full Dataset 的 Loss:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_i(\theta)$$

Mini-Batch 只用其中一部分样本:

$$L_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \ell_i(\theta)$$

不同 Batch 的样本不同，因此：

$$L_{B_1} \neq L_{B_2}$$

很正常。

所以训练曲线经常出现：

局部上上下下，但长期趋势下降

不要看到某一个 Step 的 Loss 比前一步高，就立即判断训练失败。Mini-Batch 本身引入了采样波动。应该结合更长窗口、Epoch 平均、Validation Loss 等判断。

16 一个科研细节：Epoch Mean Loss 怎样算才严谨？

假设最后一个 Batch 比其他 Batch 小。若每个 Batch 的 loss 已经是“Batch 内平均”，直接：

$$\frac{L_1 + L_2 + L_3}{3}$$

会让最后一个小 Batch 和大 Batch 拥有相同权重。

更严格的 sample-weighted 平均是：

$$L_{epoch} = \frac{\sum_b L_b \cdot n_b}{\sum_b n_b}$$

其中 n_b 是第 b 个 Batch 的样本数。

代码：

```
total_loss = 0.0
total_n = 0
for xb, yb in loader:
    ...
    total_loss += loss.item() * len(xb)
    total_n += len(xb)
epoch_loss = total_loss / total_n
```

这是典型的“代码看起来能跑，但统计口径可能不同”的科研问题。比较论文结果时，不只是看 Loss 名字，还要看 reduction、batch aggregation、masking 和 denominator 到底是什么。

17 num_workers: DataLoader 能不能并行取数据？

DataLoader 还可以：

```
DataLoader(dataset, num_workers=4)
```

让多个 worker 并行准备 Sample。

这对：

- 图像解码；

- 大量磁盘读取；
- 较重的数据增强；

可能很有帮助。

但本课程早期实验默认：

```
num_workers=0
```

这样最容易调试。

Windows 初学者特别注意

Windows 使用多 worker 时，通常应把启动代码放进：

```
if __name__ == "__main__":  
    main()
```

否则可能出现子进程重复启动等问题。

不要为了“看起来专业”盲目把 num_workers 调很大。最佳值与 CPU、磁盘、数据处理成本、GPU 速度都有关。先测数据加载是不是瓶颈，再优化。

18 pin_memory 与 GPU：先理解边界

当训练在 CUDA GPU 上进行时，常见写法：

```
loader = DataLoader(..., pin_memory=True)  
  
for xb, yb in loader:  
    xb = xb.to(device)  
    yb = yb.to(device)
```

需要特别注意：

- DataLoader 默认产生 CPU Tensor；
- pin_memory=True 不等于“自动放到 GPU”；
- 真正迁移一般仍需要 .to(device)。

本讲不深入 pinned memory 性能机制，只先建立正确边界。

19 可变长度文本为什么需要 `collate_fn`?

前面的 Dataset 每个 Sample Shape 完全一致，所以默认 `collate` 可以直接 `stack`。

但文本可能是：

[12, 53, 8, 91]

和：

[4, 77]

长度不同，无法直接堆成矩形 Tensor。

因此 LLM 数据管线常需要：

Tokenizer → Padding / Truncation → Attention Mask → Batch

最终可能得到：

$input_ids.shape = (B, T)$

$attention_mask.shape = (B, T)$

$labels.shape = (B, T)$

自定义 `collate_fn` 就是告诉 `DataLoader`：

> “拿到一组 Sample 后，具体应该怎样把它们拼成 Batch?”

Dataset 定义“一个样本”；collate 定义“多个样本怎样合成一个 Batch”。这是两个不同层次。

20 与 LLM / Transformer 的直接连接

以后 LLM 训练代码可能看到：

```
for batch in train_loader:
    input_ids = batch["input_ids"].to(device)
    labels = batch["labels"].to(device)
    attention_mask = batch["attention_mask"].to(device)

    outputs = model(
        input_ids=input_ids,
        attention_mask=attention_mask,
        labels=labels,
    )
    loss = outputs.loss
    loss.backward()
    optimizer.step()
```

这里：

- Dataset：提供单条文本/Token 序列；

- DataLoader：组成 B 条序列；
- collator：处理 padding、mask、labels；
- Model：把 (B, T) Token IDs 变成内部 (B, T, D) 表示；
- Cross Entropy：对有效 Token 计算 next-token loss。

你以前在 LLM 课程里见过 Batch、Token、Attention Mask。现在应该把它们重新放回 PyTorch 的数据管线理解。

21 与 Agent Systems 科研的关系

Agent Systems 不一定训练基础模型，但仍常遇到 Dataset / DataLoader：

- 离线 Agent trajectory；
- Tool-use trace；
- 多智能体通信记录；
- 成功/失败 episode；
- Preference pairs；
- Agent benchmark 任务集合；
- 学习一个小型 reward / verifier / classifier。

一个 Sample 可能不再是简单 (x, y) ，而是一个 dict：

```
{
  "prompt": ...,
  "trajectory": ...,
  "tool_calls": ...,
  "reward": ...,
  "success": ...
}
```

做 Agent Systems 实验时，“统计单位”尤其重要。一个 trajectory 中可能包含很多 turn、很多 tool call。如果把 turn 当独立样本，却在 train/test 中让同一 trajectory 的不同 turn 泄漏到两边，会产生 Data Leakage。

22 常见误区与错误

误区 1：Batch Size=32 表示 Dataset 只有 32 个样本

错。它只表示“一次训练 Step 取 32 个样本”。

误区 2：一个 Epoch 等于一个 Step

只有当整个 Dataset 恰好一个 Batch 时才可能如此。

误区 3：shuffle 会改变 Label 与 Feature 的对应关系

正常 DataLoader 是按 Sample 一起打乱，所以 (x, y) 对应关系保持不变。

误区 4：DataLoader 会自动把 Tensor 放 GPU

通常不会。一般仍要：

```
xb = xb.to(device)
yb = yb.to(device)
```

误区 5：最后一个 Batch 永远等于 batch_size

如果 N 不能整除 B ，且 `drop_last=False`，最后一个 Batch 更小。

误区 6：Epoch Loss 直接平均 Batch Loss 永远严格正确

当最后 Batch 大小不同，而且 Batch Loss 是 mean reduction 时，简单平均每个 Batch 的 mean 会改变样本权重。

误区 7：训练/验证/测试可以先混在一起再随机取 Batch

这是严重的数据泄漏风险。应先定义 Split，再分别构造 Dataset / DataLoader。

23 【实验】本讲到底验证了什么？

实验采用“概念 -> 可检验假设 -> 对照 -> 结果”的方式。

实验 A：DataLoader 是否只是自动化手工切 Batch？

假设：`shuffle=False`，`drop_last=False` 时，DataLoader 与顺序切片结果一致。

结果：完全一致，返回 True。

实验 B：shuffle 是否改变样本集合？

假设：顺序改变，但排序后仍是原来的 0-9。

结果：成立。

实验 C: drop_last 的作用是否只在最后 Batch?

结果:

$$[4, 4, 2] \rightarrow [4, 4]$$

实验 D: Epoch / Step 计数是否符合公式?

$$N = 23, B = 6 \Rightarrow 4 \text{ batches/epoch}$$

3 Epoch 实测累计 12 Step。

实验 E: DataLoader 能否真正接进 Training Loop?

80 个样本、Batch Size 8:

$$10 \text{ steps/epoch}$$

20 Epoch:

$$200 \text{ steps}$$

模型最终学习到:

$$w = 3, \quad b \approx -2$$

24 【本讲总结】把四层职责分开

请把下面四层彻底分开:

Dataset: 一个样本是什么?

DataLoader: 怎样取样并组成 Batch?

Model/Loss: 这一批数据算出什么 Loss?

Optimizer: 根据 Gradient 怎样更新参数?

完整训练链:

$Dataset \rightarrow DataLoader \rightarrow Mini\text{-}Batch \rightarrow Forward \rightarrow Loss \rightarrow Backward \rightarrow Optimizer \text{ Step}$

与后续课程的连接

本讲已经让 PyTorch 基础训练闭环真正完整。下一阶段开始进入“训练为什么这样设计”:

- 第 33 讲: Gradient Descent;
- 第 34 讲: SGD 与 Mini-Batch SGD;
- 第 35 讲: Batch / Epoch / Step 的训练统计与进一步细化;
- 第 36 讲: Learning Rate;

- 第 37 讲: Momentum;
- 第 38 讲: Adam;
- 第 39 讲: AdamW;
- 第 40 讲: Initialization、Vanishing/Exploding Gradient。

到这里,“模型怎么拿到数据、怎么前向、怎么算 Loss、怎么反传、怎么更新参数”的最小 PyTorch 闭环已经建立。接下来不再只是会运行 Training Loop,而要理解为什么不同优化策略会产生不同训练行为。